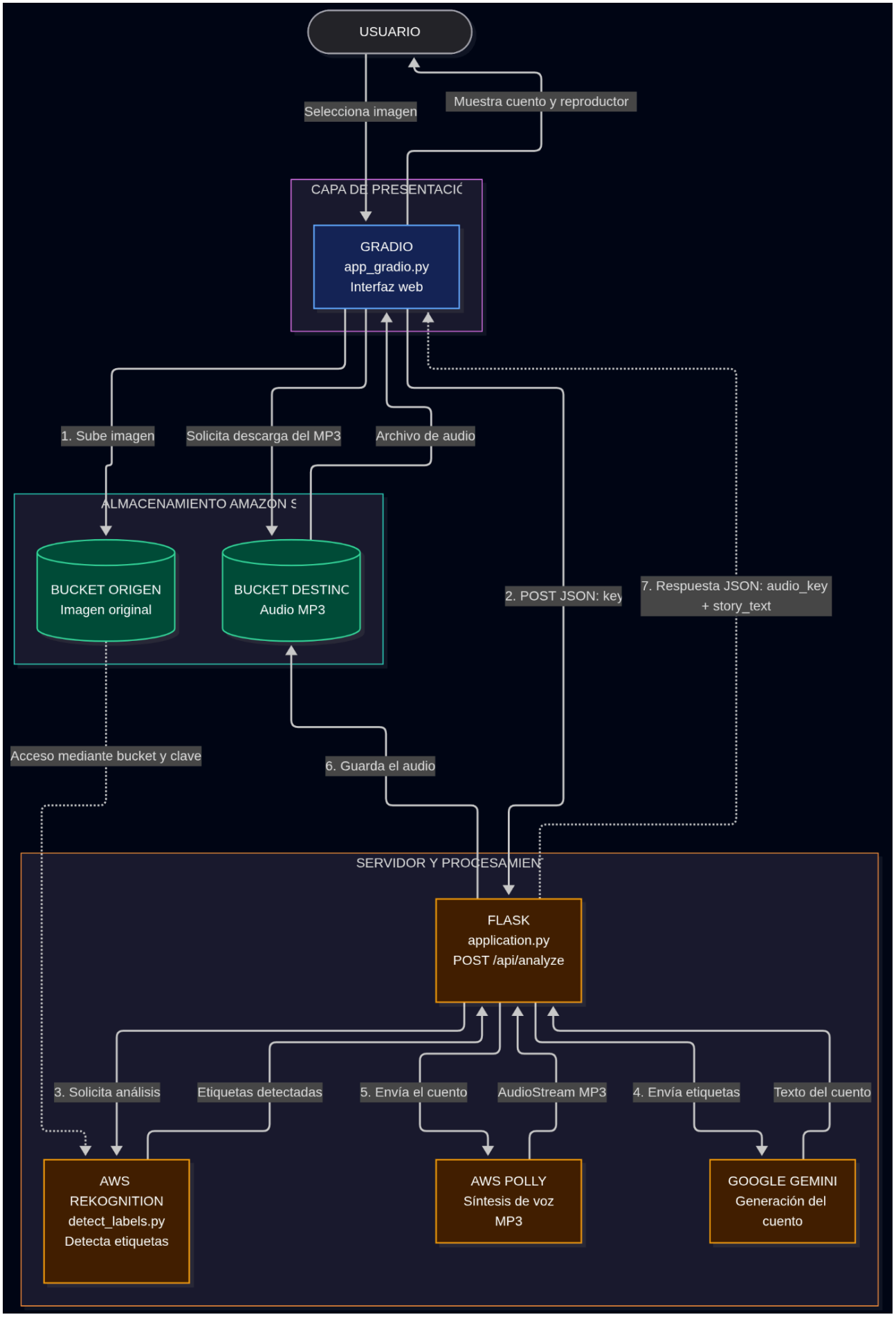
Arquitectura del Sistema:
El Cuentacuentos Visual (AWS + IA)

1. Descripción del Flujo de Trabajo General

El sistema propuesto, denominado "El Cuentacuentos Visual", es una aplicación basada en inteligencia artificial que permite a un usuario subir una imagen, analizar su contenido y generar automáticamente un micro-cuento en formato de audio, dictado en español, inspirado en los elementos presentes en la fotografía.

El flujo de trabajo sigue los siguientes pasos:

1. **Entrada del usuario**: El usuario accede a la interfaz gráfica web (desarrollada con Gradio) y sube una fotografía.
2. **Carga y Comunicación**: La interfaz web aloja la imagen temporalmente en un nodo de almacenamiento origen de AWS (bucket S3). Tras esto, envía una petición POST al servidor API (Flask) proporcionando el nombre del archivo.
3. **Análisis Visual (IA)**: El servidor invoca el servicio de análisis de imágenes de Amazon (Rekognition) para detectar las etiquetas (objetos o conceptos) presentes en la fotografía.
4. **Generación de Texto (IA Extra)**: Las etiquetas extraídas se envían a un Gran Modelo de Lenguaje (LLM), en este caso Google Gemini, instruyéndole para que redacte un cuento infantil corto en español basado en dichos conceptos.
5. **Síntesis de Voz (IA)**: El texto generado se envía al servicio de texto a voz de Amazon (Polly), el cual devuelve un flujo de audio en formato MP3 simulando una locución humana.
6. **Almacenamiento y Respuesta**: Una vez generado el audio, el servidor Flask lo almacena en el bucket de destino de Amazon S3 en formato MP3. Posteriormente, devuelve a la interfaz Gradio una respuesta JSON con el nombre del archivo de audio (audio_key) y el texto del cuento generado (story_text), junto con un mensaje de confirmación del proceso.
7. **Salida**: La interfaz Gradio recibe la respuesta JSON del servidor Flask y extrae el texto del cuento (story_text) y la clave del archivo de audio (audio_key). A continuación, utiliza esta clave para descargar el MP3 desde el bucket de destino de Amazon S3, guardándolo en una ubicación temporal del sistema. Finalmente, muestra al usuario el texto del cuento generado y carga el archivo de audio en un reproductor integrado en la interfaz, desde el que puede escuchar la narración.


USUARIO
Selecciona imagen
Muestra cuento y reproductor
CAPA DE PRESENTACIÓ
GRADIO
app_gradio.py
Interfaz web
1. Sube imagen
Solicita descarga del MP3
Archivo de audio
ALMACENAMIENTO AMAZON S
BUCKET ORIGEN
Imagen original
BUCKET DESTINC
Audio MP3
2. POST JSON: key
7. Respuesta JSON: audio_key
+ story_text
Acceso mediante bucket y clave
6. Guarda el audio
SERVIDOR Y PROCESAMIEN
FLASK
application.py
POST /api/analyze
3. Solicita análisis
Etiquetas detectadas
5. Envía el cuento
AudioStream MP3
4. Envía etiquetas
Texto del cuento
AWS
REKOGNITION
detect_labels.py
Detecta etiquetas
AWS POLLY
Síntesis de voz
MP3
GOOGLE GEMINI
Generación del
cuento

2. Módulos de la Arquitectura

Comunicación

El sistema emplea un modelo cliente-servidor basado en una arquitectura REST, utilizando el framework Flask. La comunicación permite mantener los servicios desacoplados: la interfaz envía la información en formato JSON ({"key": "nombre_imagen.jpg"}) mediante una petición POST al endpoint /api/analyze , y el servidor gestiona el resto del ciclo de vida sin necesidad de conocer los detalles del cliente.

Servicios Web (API REST)

El servidor Flask expone un único endpoint:

POST /api/analyze: Este servicio actúa como controlador principal del sistema. Recibe una petición HTTP POST con un objeto JSON que contiene la clave de la imagen (key), organiza las llamadas secuenciales a los módulos de Rekognition, Gemini y Polly, gestiona los posibles errores de ejecución y almacena el audio generado en el bucket de destino de S3. Una vez completado el proceso, devuelve una respuesta HTTP 200 OK con un objeto JSON que contiene tres campos: message, que indica que el proceso se ha completado; audio_key, que identifica el archivo MP3 almacenado en el bucket de destino; y story_text, que contiene el cuento generado por Gemini. Es importante señalar que Flask no devuelve una URL directa del audio, sino la clave del objeto almacenado en S3, que posteriormente utiliza la interfaz Gradio para descargarlo.

```JSON
{
  "message": "Proceso completado",
  "audio_key": "story_campamento.mp3",
  "story_text": "El cuento generado por Gemini..."
}
```

Servicios de IA

El proyecto integra una arquitectura multi-modelo que combina servicios de AWS y LLMs externos:

Análisis de Imágenes (Amazon Rekognition): Se ha empleado la funcionalidad detect_labels para identificar objetos, conceptos y escenarios dentro de la imagen, filtrando únicamente aquellos con una confianza superior al 80%.

Procesamiento de Lenguaje Natural (Google Gemini 3.5 Flash): Se utiliza la API de Google Generative AI para añadir una funcionalidad creativa avanzada. Recibe una lista de palabras y genera un micro-cuento estructurado en lenguaje natural.

Texto a Voz (Amazon Polly): Se utiliza la funcionalidad synthesize_speech para convertir el texto del cuento generado por Gemini en un archivo de audio MP3. Para ello, se emplea la voz Lucía (VoiceId='Lucia'), correspondiente al español, utilizando el motor de síntesis estándar (Engine='standard').

Tareas

El sistema encapsula la lógica de negocio en módulos independientes (`.py`) para fomentar la cohesión y el mantenimiento:

`application.py` : Archivo principal del servidor Flask. Recibe las peticiones POST, comprueba los datos recibidos, coordina la llamada a Rekognition y a la función encargada de generar el cuento y el audio, guarda el MP3 resultante en el bucket de destino y devuelve la respuesta JSON a la interfaz.

`detect_labels.py` : Tarea encargada exclusivamente de establecer la sesión con AWS Rekognition, leer la imagen desde S3, solicitar las etiquetas y devolver una lista limpia de cadenas de texto.

`generate_audio_story.py` : Tarea que encapsula la funcionalidad extra. Gestiona tanto la creación del prompt y la llamada a Gemini, como la posterior limpieza del texto (eliminación de formato Markdown) y su envío a AWS Polly para generar el flujo binario de audio.

`app_gradio.py` : Tarea encargada del framework visual (Gradio). Maneja la subida de imágenes, las peticiones HTTP a Flask y la manipulación de archivos temporales mediante la librería nativa `tempfile` , garantizando un entorno de ejecución limpio.

Servicios de Almacenamiento

Se ha utilizado el servicio de almacenamiento simple Amazon S3 para gestionar los datos persistentes del sistema. Se emplean dos buckets distintos:

Bucket de origen: Almacena la imagen original subida por el usuario. Amazon Rekognition accede a ella mediante el nombre del bucket de S3 y la clave del objeto (nombre del archivo), que se proporcionan en la petición de análisis.

Bucket de destino: Almacena el archivo MP3 generado por Polly, permitiendo a la interfaz web descargarlo bajo demanda sin sobrecargar la memoria del servidor Flask.

3. Funcionamiento de la llamada al servicio IA de AWS (Rekognition)

La integración principal con la plataforma en la nube de Amazon se realiza mediante la biblioteca `boto3` .

Llamada al servicio: Se inicializa una sesión con credenciales seguras (obtenidas mediante variables de entorno en un archivo .env). Posteriormente, se invoca el método detect_labels del cliente

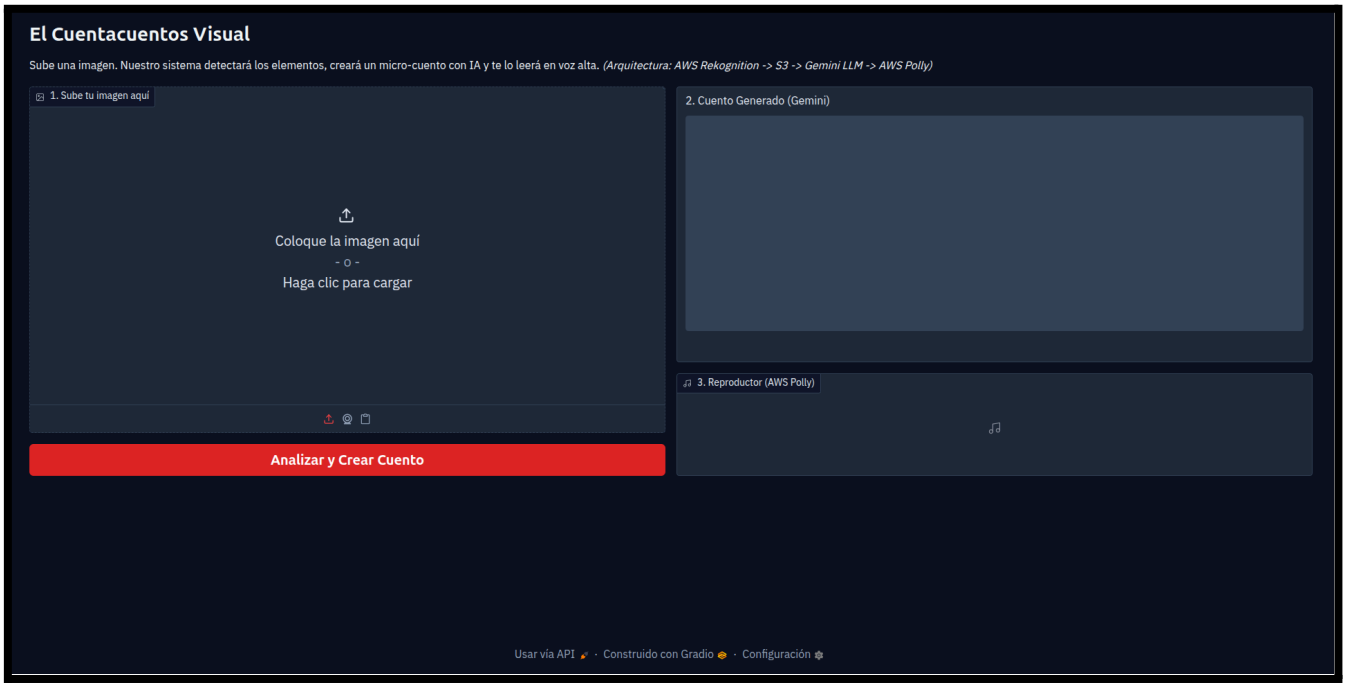
Rekognition, pasándole como argumentos el nombre del bucket S3 y la clave (nombre del archivo) de la imagen a analizar. Se ha optimizado la llamada limitando el número máximo de etiquetas (MaxLabels=7) y estableciendo un umbral de confianza mínimo (MinConfidence=80).

Respuesta del servicio: La API de AWS devuelve un objeto JSON estructurado que contiene una lista denominada Labels . Cada elemento de esta lista incluye el nombre de la etiqueta detectada (ej. 'Dog', 'Park'), el nivel de confianza algorítmica (Confidence), y en algunos casos, instancias específicas con sus cuadros delimitadores (BoundingBox) y categorías padre. Para este sistema, el código procesa iterativamente este JSON extrayendo únicamente el valor de la propiedad Name de cada etiqueta, generando una lista plana de palabras que será utilizada posteriormente por el LLM.

4. Ejemplo de uso de la aplicación documentado

Paso 1: Interfaz de usuario (Subida de datos)

El usuario inicia la aplicación accediendo a la URL local proporcionada por Gradio (ej. http://127.0.0.1:7860). Visualiza una interfaz en modo oscuro dividida en dos columnas. En la columna izquierda, arrastra y suelta una fotografía (por ejemplo, una imagen que muestra un paisaje de montaña con un cielo estrellado y una tienda de campaña) y presiona el botón "Analizar y Crear Cuento".



Paso 2: Procesamiento Interno

1. Gradio sube la imagen campamento.jpg al bucket de S3 origen y envía el JSON {"key": "campamento.jpg"} al servidor Flask local.

2. Flask llama a Rekognition, que devuelve la lista: ["Mountain", "Night", "Tent", "Stars"] .

3. Flask envía estas palabras a Gemini mediante la función encapsulada. Gemini devuelve un cuento:



4. El sistema limpia el texto de caracteres especiales y lo envía a AWS Polly, obteniendo un flujo MP3.

5. El MP3 se guarda en el bucket destino de S3 como story_campamento.mp3 .

Paso 3: Resultado Visual y Auditivo

La interfaz de Gradio recibe la confirmación del servidor, descarga el archivo story_campamento.mp3 en la carpeta temporal del sistema y actualiza la columna derecha. El usuario puede ahora leer el cuento generado por la IA en la caja de texto y hacer clic en el botón Play del componente de audio para escuchar la historia narrada con voz en español, completando así el ciclo de la aplicación

